

TRAVAUX PRATIQUES

FEUILLE N° 2 : ÉTUDE D'UN SCHÉMA AUX DIFFÉRENCES FINIES EN DIMENSION 1 POUR LE PROBLÈME DE POISSON AVEC CONDITIONS AUX LIMITES DE DIRICHLET

On s'intéresse au problème suivant : étant donnée une fonction f continue sur $]0, 1[$, trouver une fonction $u \in \mathcal{C}^2([0, 1])$ telle que :

$$(\mathcal{P}) \begin{cases} -u''(x) = f(x), & \forall x \in]0, 1[\\ u(0) = 0, u(1) = 0. \end{cases}$$

On considère une subdivision uniforme $(x_i)_{i \in \{0, \dots, N\}}$ de l'intervalle $[0, 1]$ où $x_i = i h$, $i = 0, \dots, N$ de pas $h = 1/N$. On approche le problème $(\mathcal{P})$ en utilisant un schéma de différences finies centrées d'ordre 2 de précision pour approcher l'équation différentielle. Ce schéma a été étudié en TD.

On est amené à résoudre le problème (P_h) suivant : trouver $U = (u_1, \dots, u_{N-1})^\top \in \mathbb{R}^{N-1}$ tel que $AU = F$ où

$$A = \frac{1}{h^2} \begin{pmatrix} 2 & -1 & 0 & \cdots & \cdots & 0 \\ -1 & 2 & -1 & \ddots & & \vdots \\ 0 & -1 & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & -1 & 0 \\ \vdots & & \ddots & -1 & 2 & -1 \\ 0 & \cdots & \cdots & 0 & -1 & 2 \end{pmatrix} \text{ et } F = \begin{pmatrix} f(x_1) \\ f(x_2) \\ \vdots \\ \vdots \\ f(x_{N-2}) \\ f(x_{N-1}) \end{pmatrix}.$$

La commande générique pour résoudre un système linéaire est la commande `mldivide` (ou `\`). Regarder dans la documentation quel est l'algorithme mis en œuvre dans le cas de la matrice A .

1 - Mettre en œuvre le schéma numérique proposé. Prendre $f : x \in [0, 1] \mapsto \sin(\pi x)$ et tracer sur une même figure la solution exacte $x \mapsto \frac{1}{\pi^2} \sin(\pi x)$ et la solution approchée pour $N = 25$ puis pour $N = 50$. Calculer l'erreur quadratique relative. Commenter les résultats obtenus.

2 - La matrice A est une matrice tridiagonale symétrique définie positive. Le système linéaire $AU = F$ peut être résolu plus efficacement qu'en utilisant la commande générique `mldivide` en exploitant la structure tridiagonale de la matrice. Vérifier qu'une matrice tridiagonale symétrique définie positive $A \in \mathcal{M}_{N-1}(\mathbb{K})$, de terme générique $a_{i,j}$, $(i, j) \in \{1, \dots, N-1\}^2$ avec $a_{i,j} = 0$ pour tous (i, j) tel que $|i - j| > 1$, admet une décomposition¹ $LDL^\top$ avec

$$L = \begin{pmatrix} 1 & & & & & \\ \ell_1 & 1 & & & & \\ & \ell_2 & 1 & & & \\ & & \ddots & \ddots & & \\ & & & \ell_{N-2} & 1 & \end{pmatrix} \text{ et } D = \begin{pmatrix} d_1 & & & & & \\ & d_2 & & & & \\ & & d_3 & & & \\ & & & \ddots & & \\ & & & & d_{N-1} & \end{pmatrix}$$

1. Du point de vue du calcul numérique, on préfère cette décomposition à la décomposition de Choleski $LL^\top$ car elle évite l'extraction de racines carrées pour le calcul des coefficients de la matrice L .

où les coefficients des matrices L et D sont donnés par

$$\begin{cases} d_1 &= a_{1,1} \\ d_i &= a_{i,i} - \ell_{i-1}^2 d_{i-1} & \text{pour } i = 2, \dots, N-1 \\ \ell_i &= a_{i,i+1}/d_i & \text{pour } i = 1, \dots, N-2 \end{cases}.$$

Résoudre le système linéaire $AU = F$ revient à résoudre successivement les deux systèmes linéaires triangulaires « creux » $LV = F$ puis $L^\top U = D^{-1}V$. La solution de chacun de ces deux systèmes linéaires peut être facilement explicitée. Mettre en œuvre cette méthode pour résoudre le système linéaire $AU = F$. On écrira une fonction MATLAB `choltrd` dont l'entête sera le suivant :

```
function x = choltrd(A,F)
% résout le système linéaire A x = F par la méthode de choleski L D L^t pour une
% matrice A tridiagonale symétrique définie positive.
```

Tester votre fonction en résolvant les systèmes linéaires de la question 1 et obtenir une approximation de la solution u du problème $(\mathcal{P})$ aux nœuds de la subdivision.

3 - Comparer, sur plusieurs exemples, le temps CPU de calcul (commande `cputime`) et la place mémoire (commande `whos`) nécessaires à la résolution du problème $(\mathcal{P})$ lorsque le système linéaire $AU = F$ est résolu

- par la commande `mldivide` pour une matrice A stockée sous forme pleine
- par la commande `mldivide` pour une matrice A stockée sous la forme « sparse » proposée par MATLAB
- par la fonction `choltrd` de la question 2.

4 - On montre que le schéma de discrétisation proposé est convergent au second ordre. Illustrer ce résultat pour l'exemple $f : x \mapsto \sin(\pi x)$.

a) On tracera la courbe donnant l'erreur quadratique relative en fonction du pas $h = \frac{1}{N}$ de la discrétisation, pour N variant de 10 à 10000 (penser à utiliser une échelle logarithmique (commande `loglog`) et construire la matrice A du système linéaire directement en stockage **sparse** pour réduire les besoins en mémoire).

```
B=[-ones(1,N-1);2*ones(1,N-1);-ones(1,N-1)]';
A=(1./h^2)*spdiags(B,[-1,0,1],N-1,N-1);
```

b) On déterminera le taux de convergence empirique sur cet exemple en calculant avec la commande `polyfit` le meilleur polynôme d'approximation de degré 1, au sens des moindres carrés linéaires, des données correspondant au logarithme de l'erreur en fonction du logarithme du pas h .

5 - La solution du problème $(\mathcal{P})$ admet pour formule de représentation intégrale

$$u(x) = xF(1) - F(x) \quad \text{où} \quad F(x) = \int_0^x \int_0^t f(s) \, ds \, dt. \quad (1)$$

On souhaite comparer l'efficacité de la méthode des différences finies à l'utilisation d'une formule de quadrature numérique pour calculer une approximation de la solution du problème $(\mathcal{P})$.

a) Écrire un programme permettant de calculer une approximation de la solution du problème $(\mathcal{P})$ grâce à la formule (1). On utilisera la fonction `dblquad` de MATLAB pour évaluer l'intégrale double.

b) Dans le cas test où $f : x \mapsto \sin(\pi x)$, représenter sur une même figure la représentation graphique de la solution exacte et de son approximation. On considérera 20 points de calcul pour obtenir une représentation assez précise de la solution approchée.

c) Comparer, grâce à l'instruction `cputime` de MATLAB, les temps de calcul de la solution approchée par la méthode des différences finies et par la méthode de quadrature. Pour avoir une comparaison réaliste, on choisira une tolérance pour la formule de quadrature de 10^{-6} et on prendra un nombre de points de subdivision N pour le schéma aux différences finies donnant une erreur relative du même ordre de grandeur.

Solutions

1 Le programme est le suivant.

```
% Résout par un schéma aux différences finies le problème aux limites
% -u''(x) = f(x) sur ]0,1[
% u(0)=0 u(1)=0
clear, close all
% données
alpha=pi;
f=@(x) sin(alpha*x); % second membre
uex=@(x) (sin(alpha*x)-x*sin(alpha))/alpha^2;
N=input('Valeur de N = '); % nombre de subdivisions de l'intervalle [0,1]
% Construction du système linéaire
h=1/N; % pas de la subdivision
x=h:h:1-h; % noeuds de la subdivision
% Construction du système linéaire
A=diag(2*ones(1,N-1))-diag(ones(1,N-2),1)-diag(ones(1,N-2),-1);
A=A./h^2;
F=transpose(f(x));
% Résolution du système linéaire
U=A\F;
% On complète le vecteur solution en ajoutant la donnée de Dirichlet en 0 et 1
U=[0;U;0];
% Calcul de la solution exacte aux noeuds
Uex=[0;transpose(uex(x));0];
x=[0,x,1];
% Affichage des résultats
plot(x,U,'.-b','LineWidth',2,x,Uex,'-r','LineWidth',2)
legend(['solution calculée pour N=',num2str(N)], 'solutionexacte', 'location','northwest');
grid
% calcul de l'erreur
disp(['erreur quadratique relative en % : ',num2str(100*norm(U-Uex)/norm(Uex))]);
```

- Dans le cas où $N = 25$, la solution calculée et la solution exacte sont représentées à la Fig. 1 à gauche. L'erreur quadratique relative vaut 0.1317%.
- Dans le cas où $N = 50$, la solution calculée et la solution exacte sont représentées à la Fig. 1 à gauche. L'erreur quadratique relative vaut 0.032905%.

On remarque qu'en passant d'une subdivision uniforme de 25 nœuds à une subdivision uniforme de 50 nœuds, l'erreur quadratique relative a été divisée par 4 (environ). Cela semble indiquer une convergence d'ordre 2 du schéma.

2 En effectuant le produit LDL^T et en égalant les coefficients de cette matrice à ceux de la matrice A , on obtient les relations proposées, ce qui établit l'existence et l'unicité d'une telle décomposition.

Pour résoudre le système linéaire $AU = F$, on résout successivement les 3 systèmes linéaires

- $LY = F$
- $DZ = Y$
- $L^T U = Z$

Chacun de ces systèmes est très simple à résoudre. On peut obtenir des formules donnant la solution de chacun de ces systèmes et ainsi programmer une méthode de résolution numérique du système linéaire $AU = F$ basé sur la décomposition LDL^T de A .

Le programme MATLAB a la forme suivante :

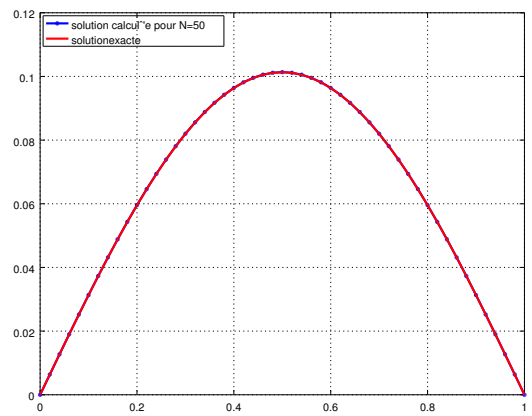
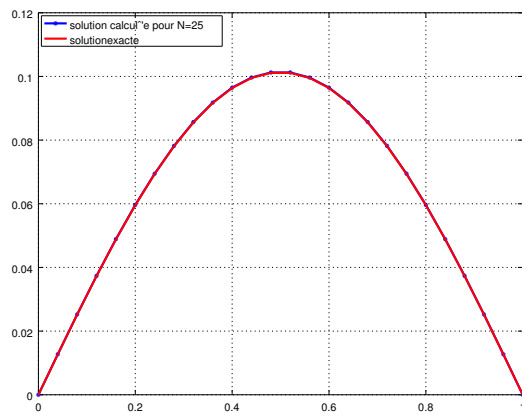


FIGURE 1 – Solution calculée et la solution exacte du problème test dans le cas d'une subdivision uniforme de 24 nœuds (à gauche) et dans le cas d'une subdivision uniforme de 50 nœuds (à droite).

```
function x = choltrd(A,F)
% résout le système linéaire A x = F par la méthode de choleski L D L^t pour une
% matrice A tridiagonale symétrique définie positive.

n=size(A,1);
% pré-allocation des tableaux pour gagner en rapidité d'exécution
d=zeros(1,n);
ell=zeros(1,n-1);
% Construction des matrices L et D
d(1)=A(1,1);
if d(1)==0, error("La matrice du système linéaire n'est pas définie positive");end
for i=1:n-1
    ell(i)=A(i,i+1)/d(i);
    d(i+1)=A(i+1,i+1)-ell(i)^2*d(i);
    if d(i+1)==0, error("La matrice du système linéaire n'est pas définie positive");end
end
% Pour test : L=diag(ones(1,n))+diag(ell(1:n-1),-1); D=diag(d); norm(A-L*D*transpose(L))
% Résolution de L y = F
y(1)=F(1);
for i=2:n
    y(i)=F(i)-ell(i-1)*y(i-1);
end
% Résolution de D z = y
z=y./d;
% Résolution de L^t x = y
x(n)=z(n);
for i=n-1:-1:1
    x(i)=z(i)-ell(i)*x(i+1);
end
x=transpose(x); % pour avoir un vecteur colonne
```

[3] Le programme MATLAB permettant la comparaison des temps de calcul est le suivant :

```

% Résout par un schéma aux différences finies le problème aux limites
% -u''(x) = f(x) sur ]0,1[
% u(0)=0 u'(1)=0
%
% Comparaison des temps de calcul entre la méthode de Gauss générique et
% la méthode de Choleski exploitant la forme tridiagonale de la matrice
%
% Auteur : Stéphane Balac - UFR de Mathématiques - Université de Rennes 1
% Décembre 2015

clear
close all
% données
f=@(x) sin(x).*exp(cos(x));
N=input('nombre de subdivisions de l\'intervalle [0,1] = ');
%
% Construction du système linéaire
h=1/N; % pas de la subdivision
x=h:h:1-h; % noeuds de la subdivision
A=diag(2*ones(1,N-1))-diag(ones(1,N-2),1)-diag(ones(1,N-2),-1);
A=A./h^2;
F=transpose(f(x));
%
% Comparaison des temps de calcul
t=cputime;
U1=A\F;
disp(['Temps CPU méthode de Matlab pour un stockage plein : ',num2str(cputime-t)]);
t=cputime;
U2=sparse(A)\F;
disp(['Temps CPU méthode de Matlab pour un stockage sparse : ',num2str(cputime-t)]);
t=cputime;
U3=choltrd(A,F);
disp(['Temps CPU méthode de Choleski pour matrice tridiagonale pleine : ',num2str(cputime-t)]);
t=cputime;
U4=choltrd(sparse(A),F);
disp(['Temps CPU méthode de Choleski pour matrice tridiagonale sparse : ',num2str(cputime-t)]);

```

On obtient les résultats suivants dans le cas où $f : x \mapsto \sin(\pi x)$ (le temps est exprimé en secondes et les calculs effectués sur un Intel core i5) :

```

>> main3
nombre de subdivisions de l'intervalle [0,1] = 5000
Temps CPU méthode de Matlab pour un stockage plein : 16.816
Temps CPU méthode de Matlab pour un stockage sparse : 0.044
Temps CPU méthode de Choleski pour matrice tridiagonale pleine : 0.272
Temps CPU méthode de Choleski pour matrice tridiagonale sparse : 0.344

```

et les résultats suivants dans le cas où $f : x \mapsto \sin(x) e^{\cos(x)}$

```

>> main3
nombre de subdivisions de l'intervalle [0,1] = 5000
Temps CPU méthode de Matlab pour un stockage plein : 17.572
Temps CPU méthode de Matlab pour un stockage sparse : 0.048
Temps CPU méthode de Choleski pour matrice tridiagonale pleine : 0.268

```

Temps CPU méthode de Choleski pour matrice tridiagonale sparse : 0.324

On voit clairement le gain que procure l'exploitation du fait que la matrice du système linéaire est tridiagonale pour la résolution du système linéaire par rapport à la résolution du système linéaire par la commande `mldivide` pour une matrice pleine. L'utilisation de la structure « sparse » de stockage des matrices creuses sous MATLAB procure un gain supplémentaire en terme de temps de calcul. Ce gain supplémentaire par rapport à notre code `choltrd.m` peut être attribué au fait que les fonctions MATLAB de résolution du système linéaire avec une matrice sparse existent en binaire sous MATLAB. Notre code `choltrd.m` doit lui être interprété par MATLAB avant d'être exécuté, ce qui ralentit l'exécution. Toutefois, des simulations tests effectuées sur d'autres machines avec d'autres versions de MATLAB ont pu montrer que le temps de calcul par la méthode de Choleski pour matrice tridiagonale pouvait être moins important que celui obtenu par la commande `mldivide` de MATLAB avec une matrice stockée sous forme sparse.

Notons que dans le cas de la méthode de Choleski programmée dans `choltrd.m`, le temps de calcul est plus important pour un stockage sparse de la matrice que pour un stockage plein. Cela s'explique par les temps d'adressage mémoire que requière le stockage sparse.

4 Le code MATLAB permettant d'illustrer la convergence du schéma est le suivant.

```
% Étude de la convergence du schéma de discrétisation par différences finis
% du problème aux limites
% -u''(x) = f(x) sur ]0,1[
% u(0)=0 u(1)=0

clear
close all
% données
alpha=pi;
f=@(x) sin(alpha*x); % second membre
uex=@(x) (sin(alpha*x)-x*sin(alpha))/alpha^2;

TN=round(10.^[1:0.5:5.5]); % nombre de subdivisions de l'intervalle [0,1]
%
err=[];
for N=TN
    % Construction du système linéaire
    h=1/N; % pas de la subdivision
    x=h:h:1-h; % noeuds de la subdivision
    B=[-ones(1,N-1);2*ones(1,N-1);-ones(1,N-1)]';
    A=(1./h^2)*spdiags(B,[-1,0,1],N-1,N-1);
    F=transpose(f(x));
    % Resolution du système
    U=choltrd(A,F);
    Uex=transpose(uex(x));
    err=[err,norm(U-Uex)/norm(Uex)];
end
% Courbe de l'erreur en fonction de N
loglog(TN,err,'o')
grid
hold on
loglog(TN,1./TN.^2,'-r','Linewidth',2);
xlabel('N')
ylabel('erreur relative');
legend('valeurs expérimentales','courbe théorique 1/N^2');

figure
```

```
% Courbe de l'erreur en fonction de h
h=1./TN;
loglog(h,err,'o')
grid
hold on
loglog(h,h.^2,'-r','Linewidth',2);
xlabel('h')
ylabel('erreur relative');
legend('valeurs expérimentales','courbe théorique  $O(h^2)$ ');
c=polyfit(log(h),log(err),1);
disp(['Taux de convergence observé : ',num2str(c(1))]);
```

La courbe donnant l'erreur l'erreur quadratique relative en fonction du pas de la subdivision est donnée à la Fig. 2. Le taux de convergence observé est de 1.6785. On voit très clairement sur cette courbe qu'en deçà de $h = 10^{-5}$ l'erreur se met à croître. Il s'agit là des effets cumulés des erreurs d'arrondis qui commencent à devenir importants pour des matrices de taille $N = 10^5$. En excluant les deux dernières valeurs, on obtient un taux de convergence empirique de 2.00063, très proche de la valeur théorique de 2.

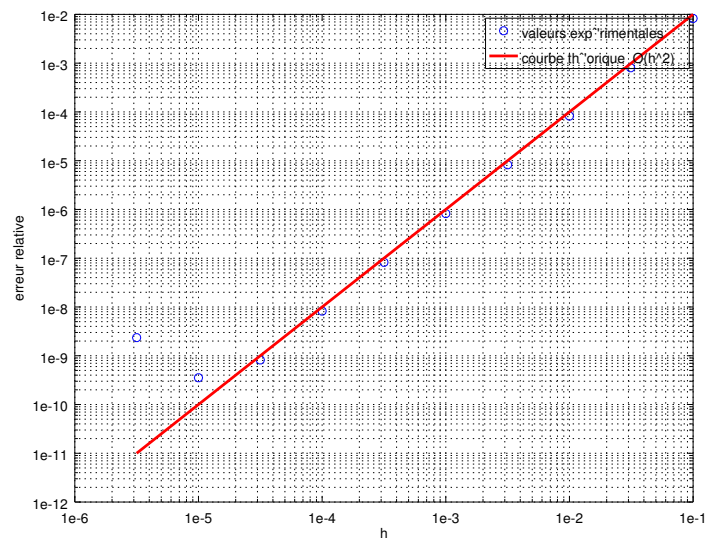


FIGURE 2 – Courbe donnant l'erreur l'erreur quadratique relative en fonction du pas de la subdivision.

5

a) Le programme `main6.m` permettant de calculer une approximation de la solution du problème ($\mathcal{P}$) grâce à la formule (1) et donnant la représentation graphique de la solution calculée et de la solution exacte dans le cas du problème test est le suivant.

```
% Calcule la solution du problème aux limites
% -u''(x) = f(x) sur ]0,1[
% u(0)=0 u(1)=0
% en utilisant la formule de représentation intégrale
%
% u(x) = x F(1) - F(x) où F(x) = int_0^x ( int_0^t f(s) ds ) dt
```

```

%
clear, close all
% données
alpha=pi;
f=@(x) sin(alpha*x); % second membre
uex=@(x) (sin(alpha*x)-x*sin(alpha))/alpha^2;
N=input('Nombre de points de calcul dans [0,1] = ');
h=1/N;
x=h:h:1-h;

g=@(t,s) f(s)*(s>0&s<t);
tol=input('Tolerance de la formule d''integration = ');
t=cputime;
C=dblquad(g,0,1,0,1,tol);
for i=1:length(x)
U(i)=x(i)*C - dblquad(g,0,x(i),0,1,tol);
end
disp(['Temps CPU =',num2str(cputime-t)]);

% On complète le vecteur solution en ajoutant la donnée de Dirichlet en 0 et 1
U=[0;U';0];
% Calcul de la solution exacte aux noeuds
Uex=[0;transpose(uex(x));0];
x=[0,x,1];
% Affichage des résultats
plot(x,U,'-ob','LineWidth',2,x,Uex,'-r','LineWidth',2)
legend(['solution calculée pour N=',num2str(N)], 'solution exacte', 'location','northwest');
grid
% calcul de l'erreur
disp(['erreur quadratique relative en % : ',num2str(100*norm(U-Uex)/norm(Uex))]);

```

b) La représentation graphique de la solution et le temps de calcul sont indiqués ci-dessous.

```

Temps CPU =11.264
erreur quadratique relative en % : 1.1796e-05

```

On obtient les temps de calcul suivants en comparant les deux approches numériques, formule de quadrature et méthode de différences finies :

```

>> main7
Valeur de N = 2000
Temps CPU de calcul =0.116
erreur quadratique relative en % : 2.0562e-05

```

La méthode par quadrature avec 20 points de calcul est 100 fois plus lente que la méthode de différences finies à précision comparable pour cet exemple. La méthode par quadrature n'a d'intérêt que si on ne souhaite obtenir une approximation de la solution qu'en un nombre très faible de points. Sur cet exemple, le temps moyen de calcul pour un seul point par la méthode par quadrature est de $11.264/20 = 0.56320$, ce qui est encore supérieur au temps de calcul par la méthode de différences finies pour $N = 2000$. Le programme `main7.m` est le suivant :

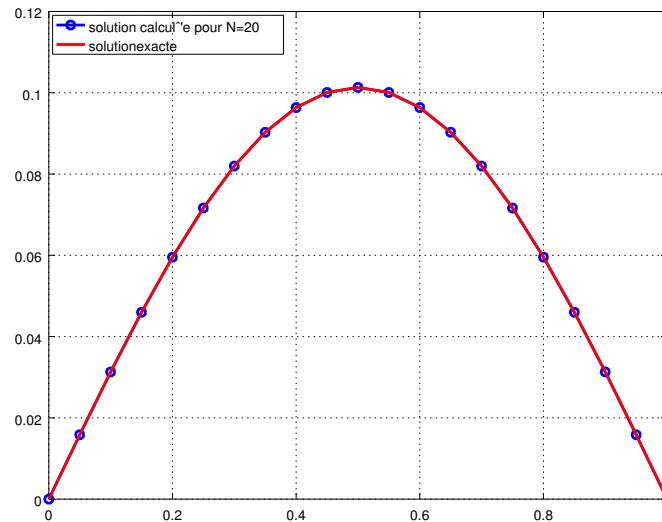


FIGURE 3 – Solution exacte du problème test et solution calculée par intégration numérique représentée avec 20 points de calcul.

```
clear, close all
% données
alpha=pi;
f=@(x) sin(alpha*x); % second membre
uex=@(x) (sin(alpha*x)-x*sin(alpha))/alpha^2;
N=input('Valeur de N = '); % nombre de subdivisions de l'intervalle [0,1]
% Construction du système linéaire
h=1/N; % pas de la subdivision
x=h:h:1-h; % noeuds de la subdivision
t=cputime;
% Construction du système linéaire
B=[-ones(1,N-1);2*ones(1,N-1);-ones(1,N-1)]';
A=(1./h^2)*spdiags(B,[-1,0,1],N-1,N-1);
F=transpose(f(x));
% Resolution du système
U=choltrd(A,F);
disp(['Temps CPU de calcul =',num2str(cputime-t)]);
% On complète le vecteur solution en ajoutant la donnée de Dirichlet en 0
U=[0;U;0];
% Calcul de la solution exacte aux noeuds
Uex=[0;transpose(uex(x));0];
x=[0,x,1];
% calcul de l'erreur
disp(['erreur quadratique relative en % : ',num2str(100*norm(U-Uex)/norm(Uex))]);
```